Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información
Centro Regional de Mazatenango
Curso: Análisis y Diseño de Sistemas II
Catedrático: Ing. Angel Atilio Maltez C.

Fase de Diseño

Sistema de Gestión del Asilo de Ancianos «Cabeza de Algodón»

Proyecto final · actualizada a la línea base ERS v3.0

Gilmer Iván Ortiz Samayoa
Carné: 3090-23-12685

Mazatenango, Suchitepéquez, 30 de octubre de 2026

Contenido

1. Planificación del trabajo....3
1.1 Metodología....3
1.2 Recursos....3
1.3 Dedicación y capacidad....3
1.4 Plan de iteraciones....3
1.5 Hitos de revisión SQA....4
2. Arquitectura....5
2.1 Estilo arquitectónico....5
2.2 Vista lógica....5
2.3 Entornos....6
2.4 Nube o servidor propio....6
2.5 Pasos de despliegue....6
2.6 Disponibilidad, rendimiento y respaldos....7
2.7 Seguridad en la arquitectura....7
3. Componentes de software....8
3.1 Vista de componentes....8
3.2 Módulo de acceso y usuarios....9
3.3 Modelo de dominio....10
3.4 Servicios y reglas de negocio....10
4. Experiencia de usuario....12
4.1 Usuarios y contexto de uso....12
4.2 Principios de diseño....12
4.3 Arquitectura de información y navegación....13
4.4 Patrones de interfaz....13
4.5 Estados visibles....14
4.6 Patrones de diseño de software....14

1. Planificación del trabajo

1.1 Metodología

El proyecto lo desarrolla una sola persona, por lo que se eligió un **modelo incremental con iteraciones de dos semanas**. Al final de cada iteración queda una parte del sistema funcionando y revisable, en lugar de probar todo al final. Cada iteración repite cuatro pasos: **requerimiento** (se eligen de la ERS los RF a trabajar), **desarrollo** (código y script SQL), **prueba** (casos de prueba con evidencia) y **revisión SQA**.

Definición de terminado. Un entregable está listo cuando tiene su requerimiento escrito en la ERS, el código y el script SQL al día, su caso de prueba ejecutado con evidencia y una revisión SQA sin hallazgos importantes pendientes.

Control de cambios. Todo cambio de alcance se anota como versión nueva en la tabla de control de versiones de la ERS, indicando qué requerimientos, casos de uso, tablas y pruebas afecta. Lo aprobado entra en la siguiente iteración; lo demás queda para la fase 2.

1.2 Recursos

Recurso	Detalle
Humano	Gilmer Iván Ortiz Samayoa: análisis, diseño, desarrollo, pruebas y documentación. Revisión de calidad: analista SQA del curso.
Equipo	Computadora personal para desarrollo; computadora del asilo o hosting para el despliegue.
Software	XAMPP (Apache, PHP 8, MariaDB, phpMyAdmin), navegador Chrome/Edge/Firefox, Python 3 con Playwright, Microsoft Word y Excel. Repositorio en GitHub para publicar el código.
Información	Necesidades del asilo, informe preliminar de revisión SQA (hallazgos H-01 a H-10).

1.3 Dedicación y capacidad

La disponibilidad es de **15 a 18 horas semanales** durante 13 semanas (24/08/2026 al 20/11/2026), con un total planificado de **230 horas**: 54 de pruebas y 26 de revisión SQA repartidas en todas las iteraciones, y 4 de holgura en la iteración 6. El archivo Planificacion_Asilo_Cabeza_de_Algodon_v3.xlsx comprueba la suma con una fórmula.

1.4 Plan de iteraciones

Iteración	Fechas	Contenido	Horas
I1	24/08 – 04/09	Análisis y línea base	35
I2	07/09 – 18/09	Modelo de datos y prototipo de pantallas	35
I3	21/09 – 02/10	Seguridad, usuarios, residentes y habitaciones	35
I4	05/10 – 16/10	Atención médica: ficha, signos, citas, laboratorio, medicamentos	35
I5	19/10 – 30/10	Farmacia, facturación, reportes, panel y API MySQL; atención del informe SQA	35
I6	02/11 – 13/11	Despliegue, pruebas de aceptación y cierre de hallazgos	35
Cierre	16/11 – 20/11	Manuales, documentación final y exposición	20

Iteración	Fechas	Contenido	Horas
Total	13 semanas		230

1.5 Hitos de revisión SQA

Hito	Fecha	Qué se revisa	Criterio para aprobar
SQA 1	04/09	Alcance y ERS	Cada RF es claro, único y verificable; alcance v1 y fase 2 definidos
SQA 2	18/09	Modelo de datos	El script SQL coincide con el diccionario de datos
SQA 3	02/10	Seguridad y residentes	PV-01 a PV-18 y CP-01 a CP-07 aprobadas
SQA 4	16/10	Atención médica	CP-08 a CP-10 y CP-15 a CP-18 aprobados
SQA 5	30/10	Administración y hallazgos	Respuesta a H-01 a H-10 con evidencia
SQA 6	13/11	Verificación en ejecución	Hallazgos cerrados por el analista SQA
Cierre	20/11	Documentación final	Control de versiones actualizado

2. Arquitectura

2.1 Estilo arquitectónico

Cliente-servidor en **tres capas** (presentación, lógica de negocio y datos). La comunicación entre la página y el servidor es HTTP con cuerpos JSON hacia un único punto de entrada, api/api.php, que recibe la acción en el parámetro accion.

2.2 Vista lógica

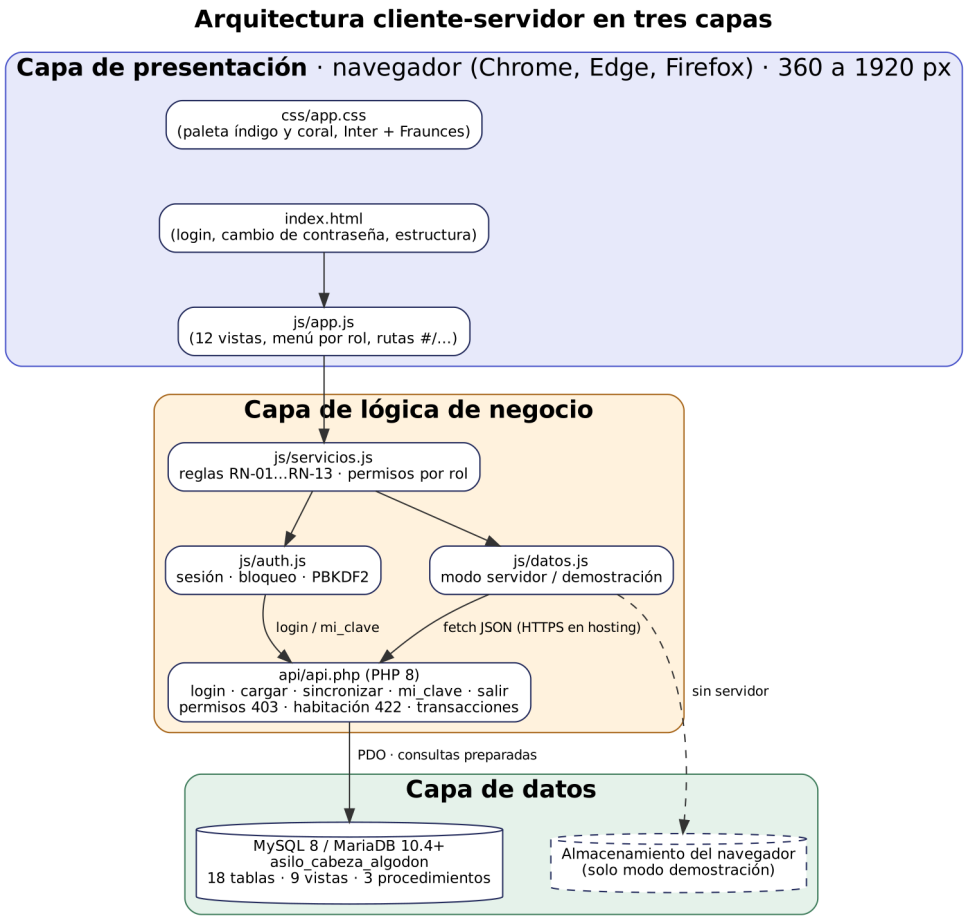


Figura 1. Vista lógica en tres capas

Acción de la API	Qué hace	Quién puede
estado / publico	Indica si hay base de datos disponible	Cualquiera
login	Valida usuario y contraseña con PBKDF2, aplica el bloqueo y abre la sesión	Cualquiera
verificar	Devuelve la sesión actual	Con sesión
mi_clave	Cambia la contraseña propia o la temporal	Con sesión
cargar	Entrega los datos permitidos para el rol	Con sesión y contraseña no temporal

Acción de la API	Qué hace	Quién puede
sincronizar	Guarda altas, cambios y bajas dentro de una transacción; revisa permisos y reglas de servidor	Según la tabla y el rol
vaciar	Restablece los datos iniciales	Solo Administrador
salir	Destruye la sesión	Con sesión

2.3 Entornos

Entorno	Uso	Configuración
Demostración	Practicar y presentar sin servidor	Abrir index.html; datos de prueba en el navegador
Desarrollo y pruebas	Programar y ejecutar las pruebas automatizadas	XAMPP local con la base importada desde 01, 02 y 03 .sql
Producción	Uso real del asilo	XAMPP en la red local u hosting con HTTPS (scripts de basedatos/hosting)

2.4 Nube o servidor propio

Se dejan preparadas las dos opciones. El **servidor propio** (XAMPP en una computadora del asilo) no tiene costo mensual y funciona sin Internet dentro de la red local, pero depende de que ese equipo esté encendido y respaldado. El **hosting** da acceso desde cualquier lugar con HTTPS y respaldo del proveedor, a cambio de una mensualidad. La decisión final corresponde a la administración del asilo; la iteración 6 contempla el despliegue y su prueba (RNF-04 y RNF-08).

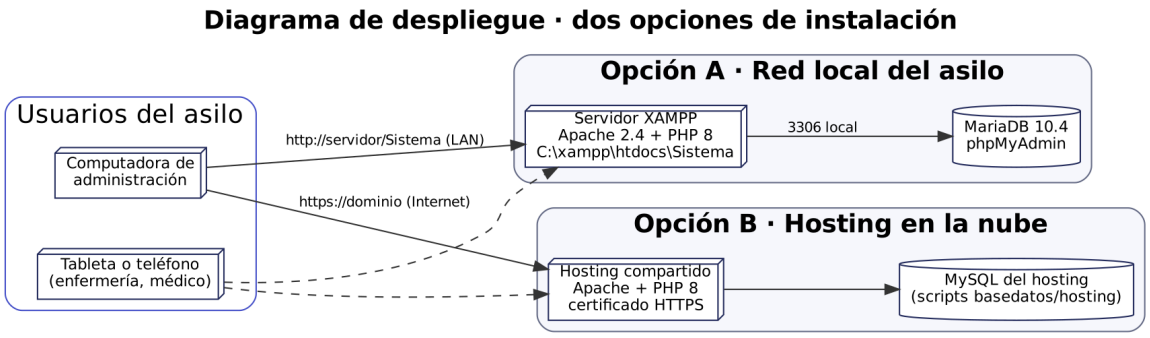


Figura 2. Arquitectura de despliegue

2.5 Pasos de despliegue

1. Copiar la carpeta Sistema en C:\xampp\htdocs (o en public_html del hosting).
2. Importar en phpMyAdmin, en orden, 01_esquema.sql, 02_datos.sql y 03_consultas.sql (en hosting, los archivos de basedatos/hosting, que no crean la base).
3. Revisar los datos de conexión en api/config.php.
4. Abrir http://localhost/Sistema/ (o el dominio con https) y entrar con la contraseña temporal del administrador, que se entrega por separado; el sistema obliga a cambiarla.
5. Crear las cuentas del personal desde Usuarios.

2.6 Disponibilidad, rendimiento y respaldos

- Disponibilidad objetivo: lunes a sábado de 07:00 a 18:00 con 99 % del horario (RNF-04, propuesto).
- Rendimiento: búsquedas y listados en menos de 2 s con hasta 200 residentes (RNF-05); medido 0.03 s con los datos de prueba.
- Respaldos: exportación semanal desde phpMyAdmin y descarga JSON desde el módulo Usuarios (RF-24).

2.7 Seguridad en la arquitectura

- Contraseñas con PBKDF2-SHA256, 100 000 iteraciones y sal de 16 bytes; el hash lo calcula PHP y la página nunca lo recibe.
- Contraseñas iniciales temporales con cambio obligatorio; mientras tanto el servidor no entrega datos (HTTP 403).
- Bloqueo de 30 s tras 3 intentos fallidos y registro de cada intento con la IP.
- Sesión renovada al entrar (session_regenerate_id) y cierre por inactividad de 30 minutos en la página.
- Permisos por rol revisados en el servidor (HTTP 403) y transacciones en cada guardado.
- HTTPS obligatorio en hosting; ninguna credencial en documentos ni pantallas.

3. Componentes de software

3.1 Vista de componentes

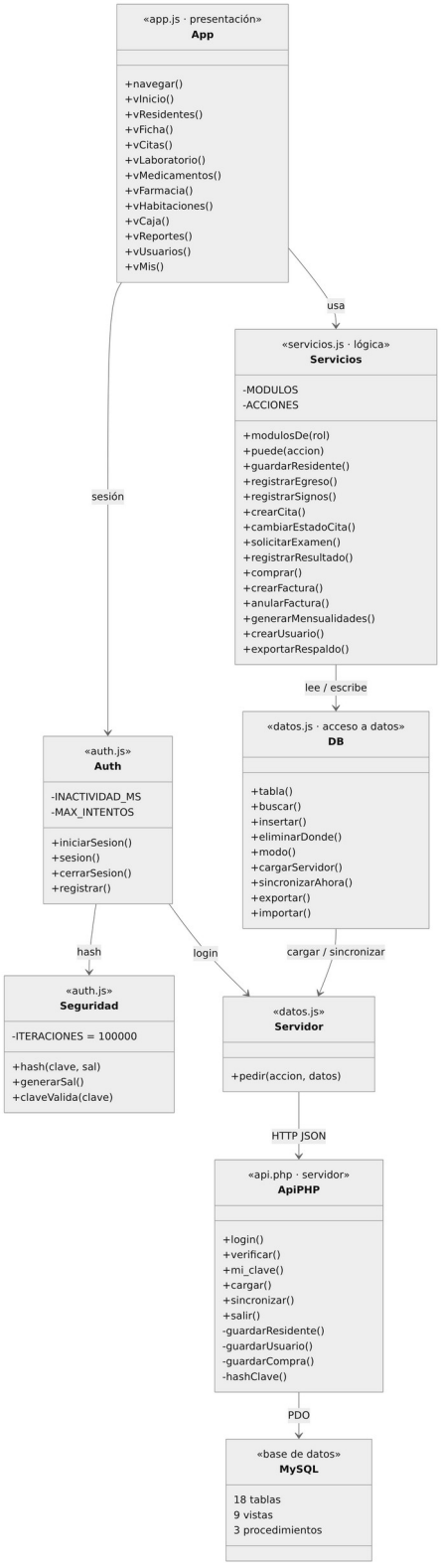


Figura 3. Componentes de software y sus dependencias

Componente	Archivo	Responsabilidad
App	js/app.js	Rutas #/módulo, menú agrupado por rol, 12 vistas, formularios, modales, avisos y notificaciones
Servicios	js/servicios.js	Reglas de negocio, validaciones, transiciones de estado y permisos (MODULOS y ACCIONES por rol)
Auth y Seguridad	js/auth.js	Inicio y cierre de sesión, inactividad, bloqueo, hash PBKDF2 y política de contraseña
DB y Servidor	js/datos.js	Datos en memoria, modo servidor o demostración, carga y sincronización con la API
API	api/api.php	Autenticación, permisos, validación de habitación, prohibición de borrar residentes, transacciones con PDO
Configuración	api/config.php	Datos de conexión a MySQL
Base de datos	basedatos/*.sql	Esquema, datos de prueba, vistas y procedimientos

3.2 Módulo de acceso y usuarios

El inicio de sesión sigue el diagrama DS-01: la página envía usuario y contraseña a la acción login; el servidor revisa el bloqueo, obtiene la sal y el hash, calcula hash_pbkdf2 y compara. Si la cuenta tiene **debe_cambiar_clave = 1**, responde que debe cambiarla y la página muestra «Crea tu contraseña personal». Solo después de cambiarla se permite la acción cargar.

3.3 Modelo de dominio

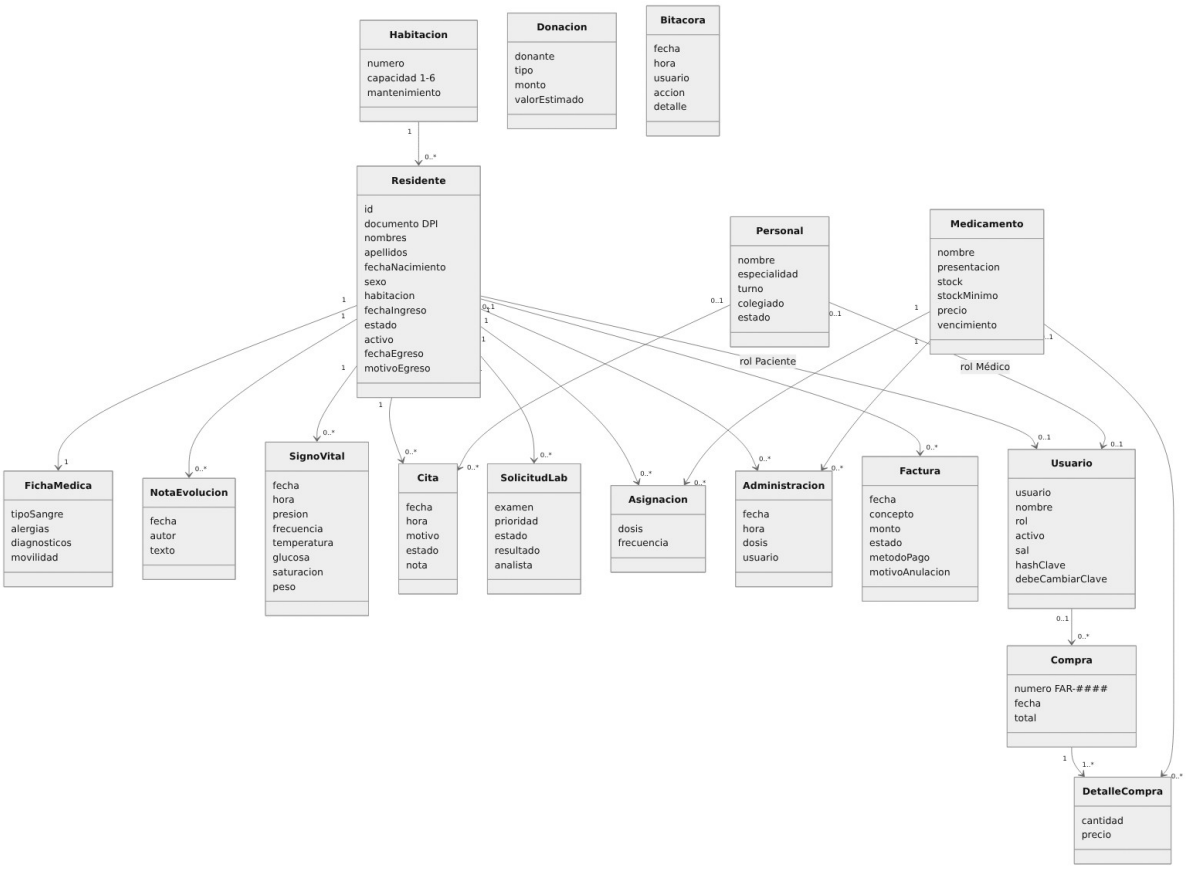


Figura 4. Modelo de dominio

3.4 Servicios y reglas de negocio

Servicio	Reglas que aplica
guardarResidente	RN-01 edad y DPI, RN-02 habitación, validación de fechas y teléfono
registrarEgreso	RN-03 baja lógica, RN-04 efectos del egreso
crearCita / cambiarEstadoCita / reprogramarCita	RN-05 horario y choques; transiciones Programada → Confirmada → En atención → Completada
solicitarExamen / iniciarExamen / registrarResultado	Prioridades y estados Pendiente → En proceso → Completado
comprar	RN-06 stock y vencimiento; recibo FAR-#### (RN-12)
crearFactura / pagarFactura / anularFactura / generarMensualidades	RN-07 y RN-08
crearUsuario / restablecerClave / cambiarMiClave	RN-09 y RN-11
exportarRespaldo / restaurarRespaldo	Respaldo JSON validado (RF-24)

4. Experiencia de usuario

4.1 Usuarios y contexto de uso

Rol	Contexto	Necesidad principal
Administrador	Oficina, computadora de escritorio	Ver el estado del asilo y registrar ingresos, cobros y usuarios rápidamente
Médico	Recorridos y consultorio, tableta o computadora	Ver la ficha completa, dar dosis y registrar signos sin perder tiempo
Laboratorio	Laboratorio, computadora	Ver la cola de solicitudes por prioridad y registrar resultados
Paciente	Teléfono o tableta	Consultar su información y comprar en la farmacia

4.2 Principios de diseño

1. **Cada rol ve solo su trabajo:** menú y panel según el rol (RNF-09).
2. **Legible para personal no técnico:** Inter para la interfaz y Fraunces para títulos, textos en español claro (RNF-02).
3. **Cualquier módulo en máximo 2 clics:** menú lateral agrupado en Principal, Atención, Administración y Sistema.
4. **El estado no depende solo del color:** cada insignia lleva texto (Estable, Pendiente, Vencido…).
5. **Prevención de errores:** validación al guardar con mensaje junto al campo; opciones no válidas deshabilitadas (habitaciones llenas).
6. **Respuesta inmediata:** avisos al terminar cada acción y notificaciones con lo pendiente.
7. **Funciona en teléfono:** desde 360 px, con barra inferior de navegación (RNF-03).

4.3 Arquitectura de información y navegación

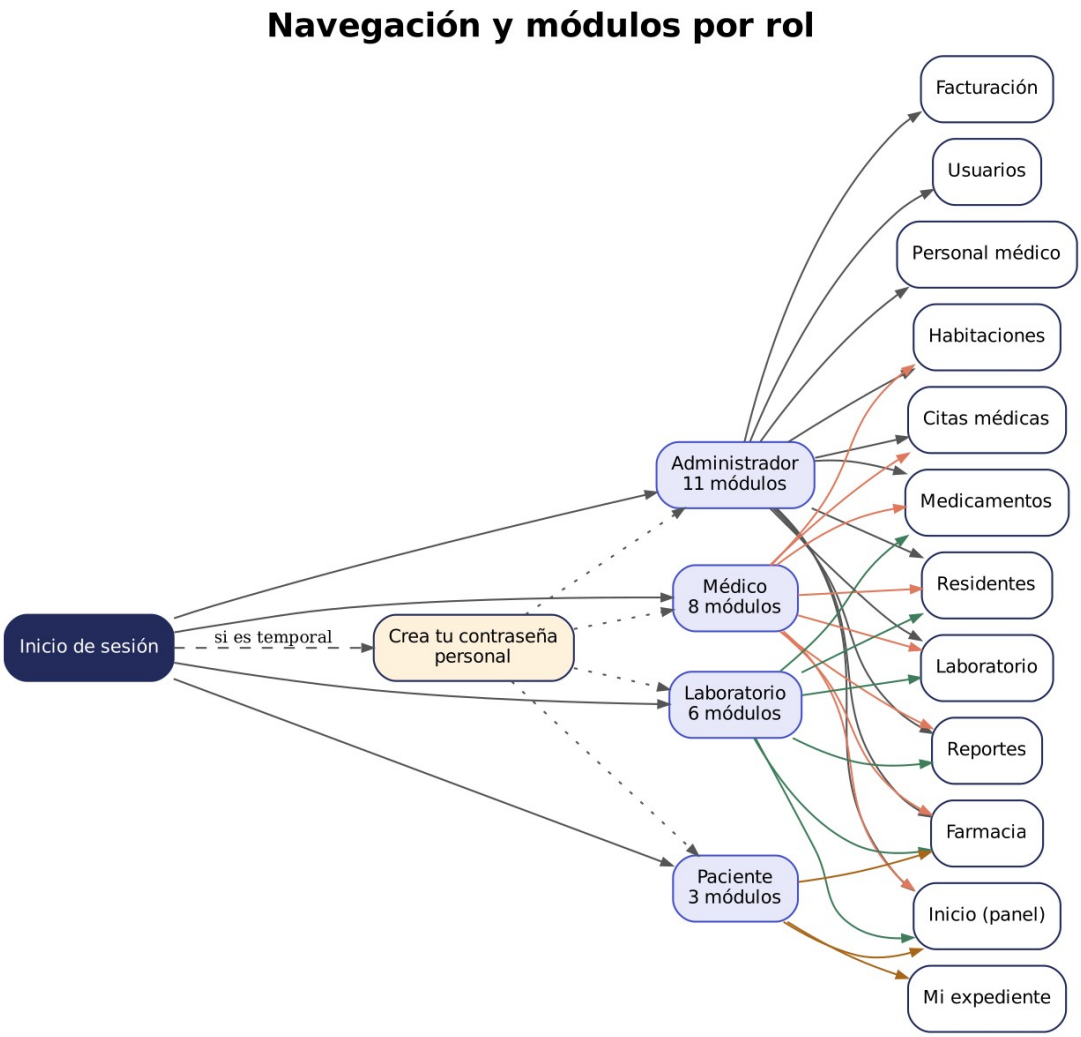


Figura 5. Navegación y módulos por rol

4.4 Patrones de interfaz

Patrón	Dónde se usa
Panel con accesos rápidos y tarjetas de indicadores	Inicio de cada rol
Tabla con buscador y filtros	Residentes, citas, personal, laboratorio, medicamentos, facturación
Ficha con encabezado del residente	Expediente: datos, ficha médica, signos, notas, medicamentos, citas y exámenes
Formulario en ventana modal	Altas y ediciones
Línea de tiempo	Agenda del día
Plano por tarjetas	Habitaciones y su ocupación
Carrito de compras	Farmacia del Paciente

Patrón	Dónde se usa
Vista previa + PDF/CSV	Reportes

4.5 Estados visibles

Las insignias usan cinco familias de color: **verde** (Estable, Completada, Pagada, Activo, Disponible), **ámbar** (En observación, Programada, Pendiente, Por vencer, Vacaciones), **azul** (Confirmada, En atención, En proceso, Ocupada), **rojo** (Delicado, Cancelada, Urgente, Vencido, Agotado) y **gris** (Egresado, Anulada, turnos). El detalle está en el Manual de Diseño Visual.

4.6 Patrones de diseño de software

Patrón	Aplicación
Módulo (IIFE de JavaScript)	Seguridad, Auth, DB, Servicios y App exponen solo sus funciones públicas
Fachada	Servicios ofrece operaciones de alto nivel (registrarEgreso) que coordinan varias tablas
Repositorio	DB centraliza el acceso a los datos, sea en memoria o en el servidor
Front controller	api/api.php recibe todas las peticiones y despacha por accion
Resultado uniforme	Cada servicio devuelve { ok, data } o { ok, error } para mostrar mensajes consistentes